



Project Report (Part I)

on

VendorHood - A Multi-Tenant SaaS Online Retail System

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

In

COMPUTER ENGINEERING

Submitted by

Nidhay Chavan

Gaurav Gautam

Aryan Goud

Under the Guidance of

Ms. Akshata Raut

Assistant Professor

Department of Computer Engineering

(Academic Year. 2026-27)



TCET THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Approved by AICTE & Govt. of Maharashtra | Affiliated to University of Mumbai (UoM)

- Conferred Autonomous Status by UGC for 10 years w.e.f. A.Y. 2019-20
- Accredited with 'A' Grade by National Assessment & Accreditation Council (NAAC)
- Programmes Accredited by National Board of Accreditation (NBA)
- Empowered Autonomous Status Conferred by University of Mumbai for 10 years w.e.f. A.Y. 2025-26



CERTIFICATE

This is to certify that the project entitled **“VendorHood – A Multi-tenant SaaS Online Retail System”** is a bonafide work of **Nidhay Chavan (Roll No. 20), Gaurav Gautam (Roll No. 44), Aryan Goud (Roll No. 48)** submitted to the Thakur College of Engineering and Technology, Mumbai (An Autonomous College affiliated to University of Mumbai) in partial fulfillment of the requirement for the **Project-I** for award of the degree of **“Bachelor of Engineering”** in **“Computer Engineering”**.

Signature with Date: _____

Signature with Date: _____

Name of Guide:

Name of HOD:

Designation:

Name of Department:



TCET THAKUR COLLEGE OF
ENGINEERING & TECHNOLOGY

Approved by AICTE & Govt. of Maharashtra | Affiliated to University of Mumbai (UoM)

- Conferred Autonomous Status by UGC for 10 years w.e.f. A.Y. 2019-20
- Accredited with 'A' Grade by National Assessment & Accreditation Council (NAAC)
- Programmes Accredited by National Board of Accreditation (NBA)
- Empowered Autonomous Status Conferred by University of Mumbai for 10 years w.e.f. A.Y. 2025-26



PROJECT APPROVAL CERTIFICATE

This project report entitled “**VendorHood – A Multitenant SaaS Online Retail System**” by *Nidhay Chavan (Roll No. 20)*, *Gaurav Gautam (Roll No. 44)*, *Aryan Goud (Roll No. 48)* is approved for the degree of “**Bachelor of Engineering**” in “**Computer Engineering**”.

Internal Examiner:

Signature: _____

Name: _____

External Examiner:

Signature: _____

Name: _____

Date:

Place:

ACKNOWLEDGEMENT

It would be unfair if I do not acknowledge the help and support given by Professors, students, friends etc.

We sincerely thank our guide Ms. Akshata Raut for her guidance and constant support and also for the stick to our backs. We also thank the project coordinators for arranging the necessary facilities to carry out the project work.

We thank the HOD, Dr. Vaishali Kaiche, the Principal, Dr. B. K. Mishra and the college management for their support.

1. Nidhay Chavan (Roll No. 20)
2. Gaurav Gautam (Roll No. 44)
3. Aryan Goud (Roll No. 48)

Blue Book Plagiarism Report



Page 1 of 56 - Cover Page

Submission ID trnoid::3618:153111942

Mrs. Lydia Suganya Blue Book VendorHood Final

 New

Document Details

Submission ID

trnoid::3618:153111942

Submission Date

Sep 28, 2026, 11:09 AM GMT+5:30

Download Date

Sep 28, 2026, 12:03 PM GMT+5:30

File Name

Blue Book VendorHood Final.pdf

File Size

2.1 MB

54 Pages

10,719 Words

66,537 Characters



Page 1 of 56 - Cover Page

Submission ID trnoid::3618:153111942

Blue Book Plagiarism Report - AI



*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.



Blue Book Plagiarism Report – Similarity



14% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **92 Not Cited or Quoted 13%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **10 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

-  **7% Internet sources**
-  **4% Publications**
-  **12% Submitted works (Student Papers)**

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

INDEX

Chapter No.		Page No.
	List of Figures	01
	List of Tables	02
	Abstract	03
Chapter 1	Introduction	
	1.1 Overview of the Project	04
	1.2 Motivation & Application	04
	1.3 Problem Definition	05
	1.4 Objective & Scope	05
	1.5 Expected outcome	06
	1.6 Organization of the Report	06
Chapter 2	Literature Survey & Proposed System	
	2.1 Literature review of Existing System	07
	2.2 Limitations of Existing System & Gap Analysis	08
	2.3 Proposed System	08
Chapter 3	Requirement Gathering, Analysis and Planning	
	3.1 Requirement Specification	12
	3.2 Feasibility Study	15
	3.3 Methodology	17
	3.4 Technology Stack	18
	3.5 Gantt Chart and Process Model	19
	3.6 System Analysis (Functional, Structural, and Behavioral Models)	20
Chapter 4	System Design and Experimental Set up	
	4.1 System Architecture & Diagrams (DFD/UML/ Block Diagram/Physical Layout)	25
	4.2 Algorithm & Process flow design (Flowchart/Pseudo Code)	26
	4.3 User Interface & Input Data Design (Snapshots/ circuits/Structure)	
	4.4 Experimental Setup and Tools (Software & Hardware)	29
	4.5 Implementation, Deployment and Testing	33
	4.6 Performance Evaluation.	34
Chapter 5	Results & Discussion	
	5.1 Outputs & Outcomes	38
	5.2 Analysis of Results & Interpretation of data	39
	5.3 Discussion of Results & Limitations of the System	40
Chapter 6	Conclusion & Future Scope	
	6.1 Summary of work completed	42
	6.2 Future Scope	43

References
Research Paper

Appendix A:	Abbreviation and symbols	45
Appendix B:	Definitions	46
Appendix C:	List of Publications	48

List of Figures

Figure 2.1: Product Entity Structure	08
Figure 2.2: Tenant-Specific Storefront Generation Process	09
Figure 2.3: Commission-Based Payment Splitting and Settlement Flow	10
Figure 2.4: Overall Proposed Multi-Tenant E-Commerce System Flow	11
Figure 3.1: Development Process Model	19
Figure 3.2: Functional Relationship	19
Figure 3.3: Tenant Isolation Structure	21
Figure 3.4: Behavioral Model	22
Figure 3.5: Storefront Template	23
Figure 3.6: Order And Commission Processing.....	24
Figure 3.7: Storefront Template.....	24
Figure 4.1: System Architecture / Physical Layout Block Diagram of VendorHood.....	26
Figure 4.2: Flowchart: Authentication and Role-Based Tenant Resolution	27
Figure 4.3: Flowchart: Multi-Tenant Checkout and Commission-Split Algorithm	28
Figure 4.4: VendorHood Landing Page	29
Figure 4.5: Tenant Management List for VendorHood Admins	29
Figure 4.6: Platform Analytics for VendorHood Admins	30
Figure 4.7: Product Catalog / Inventory for Vendors	30
Figure 4.8: Shop Analytics + Order Ledger for VendorHood Admins	31
Figure 4.9: Vendor Customizable Storefront	31

List of Tables

Table 3.1: Technology Stack	18
Table 4.1: Input Data Design Summary	32
Table 4.2: Software Requirements	33
Table 4.3: Hardware Requirements	34
Table 4.4: Implementation, Deployment and Testing	34

ABSTRACT

The proposed project, VendorHood, is a multi-tenant Software as a Service (SaaS)-based online retail platform designed to enable multiple independent vendors to operate their own customized online storefronts using shared application infrastructure. Traditional e-commerce systems often require businesses to bear separate costs for software development, hosting, customization, maintenance, and infrastructure. VendorHood addresses this challenge by adopting a multi-tenant architecture that allows vendors to share a common platform while maintaining logical separation of their data and configurations.

The system focuses on tenant identification, tenant-context enforcement, customizable storefronts, subdomain and custom-domain support, tenant-specific product catalogues, customer authentication, checkout, order management, and commission-based transaction processing. A shared-schema approach is used in which tenant-specific records are identified using a tenant ID, while every request is associated with the appropriate tenant to restrict database operations and prevent cross-tenant data access. The platform also incorporates template-based storefront customization, allowing vendors to select predefined designs without requiring separate frontend development for each tenant.

The project further aims to integrate payment splitting between the platform and vendors, domain-based tenant resolution, and vendor-specific order and catalogue management within a single retail system. These features address gaps identified in existing multi-tenant SaaS research, particularly the limited integration of tenant isolation, storefront customization, domain resolution, and commission-based payment settlement in one platform. The proposed system is intended to reduce infrastructure and operational costs while providing a scalable foundation for small businesses, startups, and multi-brand retailers seeking to establish an online presence.

Chapter 1 — Introduction

1.1 Overview of the Project

The project is a multi-tenant SaaS-based online retail platform where multiple independent online retailers can have their own storefront on the same platform infrastructure.

In a traditional e-commerce setup, every company could have to purchase its own software, hosting, customization, maintenance, and other infrastructure. This results in a high fixed cost, especially for small businesses and startups.

To solve this problem, the proposed system is based on the concept of Multi-tenancy, which is defined as sharing of application and infrastructure while maintaining separate data and configuration.

The proposed system is comprised of several key architectural elements: shared-schema multi-tenancy, tenant identification, tenant-context enforcement, template-based storefront customization, subdomain and custom-domain support, commission-based transaction processing, and splitting payment between the platform and vendors, and vendor-managed order fulfillment. This system also takes care of important issues like performance isolation and noisy-neighbor problem. In general, the goal is to offer a cost-synchronised SaaS retail platform that allows vendors to get involved in e-commerce without having to invest in a single e-commerce system.

1.2 Motivation & Application

Motivation

The major project driver is the cost involved in going digital, which could include software development, software customization, infrastructure, integration, maintenance, training, and ongoing operational expenses, among others, for small and micro-scale businesses. Such costs may need to be borne before a business has enough online sales to cover them, which presents an economic challenge to digitalisation for businesses with limited finances. A multi-tenancy can be an alternative where multiple vendors share the same application and infrastructure, but where there is still logical isolation of data and the cost of the underlying system is spread across multiple tenants. The commission-based model also aids in this goal, as the platform's income doesn't rely solely on fixed subscription costs or initial fees, but on transactions instead.

Applications

The suggested framework can be utilized to a number of business models and e-commerce models. It's ideal for small online retailers who wish to create an online storefront without having to build their own system. It can also be used to power multi-brand marketplaces, where each brand is using the same SaaS platform. Vendors can have their own tenant and

storefront on vendor-based e-commerce platforms, and retail platforms for startups can leverage the framework to get into the digital commerce space with less upfront infrastructure. Plus, the framework is designed to accommodate custom-domain storefronts, which let vendors log into and manage their stores with their own subdomains or domains.

1.3 Problem Definition

Small and micro-scale businesses encounter great technical and financial barriers when. Setting up their own e-commerce stores. Traditional e-commerce platforms can come with high upfront and ongoing expenses, From development to customization, infrastructure, maintenance, integration and more. operational expenses. These fixed costs can be a burden before a vendor has generated revenue, or before they have begun to make profit. sufficient revenue.

Meanwhile, having a common multi-tenant architecture brings technical challenges. The system should prevent one tenant from interfering with another, as multiple vendors are using the same infrastructure. Access to other tenant's data.

The problem can thus be stated as: To develop an affordable multi-tenant SaaS solution for an online retail business to enable providing multiple vendors with their own storefronts on shared infrastructure. The ability to store tenant data separately, customize the storefronts, and use domain-based tenants. Identification, commission settlement based on transactions. The System particularly identifies the risk of cross-tenant data leakage in shared-schema architectures and so puts tenant-context enforcement at its core.

1.4 Objective & Scope

Objectives

The primary goals of the proposed system are: to create a multi-tenant SaaS architecture, and to implement a multi-tenant SaaS application. providing a single application and infrastructure for multiple vendors. maintaining logical separation of their data. The system seeks to lower the fixed costs by employing shared infrastructure and a common codebase, thereby minimizing development, Operational overhead, maintenance, and. It also focuses on implementing tenant isolation by ensuring that each request is associated with the correct tenant and that database That tenant is the only tenant that will be able to use operations. Another objective is to provide customizable no code or design requirements, enabling the vendors to choose from pre-designed storefronts and layouts without coding or designing. Requiring individual development by each vendor. The system also seeks to put in place domain-based tenant resolution (both subdomain based and custom domain based) storefronts. Moreover, commission monetization will be introduced to process The platform and vendor are paid their respective portions of each payment and that the platform ensures the proper payment of each to the vendor. transaction. Lastly, the system

is designed to be scalable and performant, by Taking into account resource sharing, performance interference and the noisy-neighbor problem In a common Java-based world. These objectives correspond to the framework's

Scope

The proposed system is designed to cover vendor onboarding, tenant creation, tenant ID, and other related areas. The products are displayed in catalogues, a storefront is generated, users browse the products, check the stock, check out, and The payment splitting feature with commissions, updates to the order history, tenant data isolation.. The system is designed to support the core the functions of a multi-tenant e-commerce platform, and maintaining separation of functions. vendor data. The proposed system, however, is not based on logistics or physical order. fulfilment. The proposed system has shipment and fulfilment as the the individual vendors' responsibility.

1.5 Expected Outcome

The intended outcome of the proposed system should be an operational architectural design of one or more SaaS (Software-as-a-Service) multi-tenant online retail system that will enable multiple vendors to run over shared infrastructure. The system is supposed to deliver the following benefits: reduced infrastructure and operational costs due to resource sharing, logical separation of data between tenants, tenant-aware product and catalogue retrieval, customisable storefronts with predefined templates, support for subdomains and custom-domains, commission settlement by transaction (partial or split settlement between vendor and platform) and consistent order status updates after successful payment. It will also provide a foundation for future scalability and performance isolation. The vendor should be onboarded and then the tenant/domain provisioned, the customer should log in, browse a tenant-specific catalogue, check the stock and then checkout and go for the payment split.

1.6 Organization of the Report

The report is structured in several chapters in order to systematically present the development and analysis of the proposed multi-tenant SaaS system. The project background, motivation, problem, project objectives, scope and expected outcomes are provided in Chapter 1. Literature survey and analysis of existing system, identification of the limitations and research gaps of the existing system followed by proposed system are given in chapter 2. The following chapters explain the requirements of the system, system architecture and design, implementation methodology, test and result of the proposed system and finally, conclusion and future scope of the proposed system.

Chapter 2 — Literature Survey & Proposed System

2.1 Literature Review of Existing System

The literature review explores already present research about the multi-tenant SaaS, e-commerce platforms, tenant isolation, storefront customization, and performance metric evaluation for the same technologies.

Shaikh and Patil [1] proposed a SaaS-based multi-tenant e-commerce model that reduces development and maintenance costs through shared infrastructure and resources. Pushpan [2] presented a comprehensive multi-tenant SaaS framework covering database partitioning, tenant isolation, authentication, authorization, scalability, and security. Rathod et al. [3] studied a multi-vendor SaaS platform integrating website building, CRM, marketing automation, e-commerce, and tenant management. Kumar [4] examined multi-tenant SaaS architecture with emphasis on database partitioning, security, access control, and tenant isolation. Kalra and Prabhakar [5] proposed a dynamic tenant-management approach for microservice-based SaaS applications to improve resource utilization for heterogeneous tenants. Zhuang et al. [6] investigated performance interference in multi-tenant Java cloud applications and emphasized the need for effective tenant isolation. Shaikh and Patil [7] explored a multi-tenant e-commerce SaaS model for reducing IT costs through shared infrastructure and transaction-based charging. Bemagoni and Kare [8] presented a cloud-based multi-tenant and multi-vendor e-commerce platform. Pinto et al. [9] conducted a systematic mapping study of multi-tenant SaaS architectures and identified major challenges related to customization, migration, and architectural design. Makki et al. [10] investigated thread-level resource control for tenant-specific code in shared JVM environments to improve resource isolation. Finally, Ochei and Petrovski [11] proposed a decision-based deployment model that balances resource consumption, performance, and tenant isolation in cloud applications.

2.2 Limitations of Existing System & Gap Analysis

The missing link is the absence of an integrated retail architecture, where several studies are dedicated to general SaaS or multi-tenancy and do not integrate all the needs of an online retail platform. Additionally, there is limited integration of commission-based payment, for those few architectures that talk about multi-tenancy, they don't specifically talk about integration of commission-based payment and settlement. A limitation of using storefront customization is that many studies focus only on backend architecture and security, and less on tenant-specific frontend and storefront customization.

In addition, subdomain and tenant identification and data isolation are not fully integrated into existing work; while some may mention subdomains, they do not necessarily include both subdomain and custom-domain resolution as components of the tenant identification

and data isolation process.

The other is the lack of integration of tenancy and transaction processing, as some system concentrate on the management of tenants but don't include complete retail transaction processing, from splitting payment to updating orders. Last but not least, performance isolation continues to be a challenge: If the Java environment is distributed across tenants, then the resources of each tenant can be interfered with by another tenant, especially when it comes to CPU and memory.

Research Gap

The proposed work is aimed at filling the gap in the literature, by addressing the combination of: All of these translate to: multi-tenancy + tenant isolation + customizable storefronts + domain resolution + commission-based payment settlement In a single online retail SaaS architecture.

2.3 Proposed System

The proposed system is a multi-tenant SaaS-based online retail framework where multiple vendors operate their individual storefronts through a common platform.

The system consists of four major architectural areas:

1. Data Architecture

The system uses a shared-schema multi-tenant model.

Tenant-specific records are stored in common tables and identified using a tenant ID/discriminator.

For example:

Product

Product ID
Tenant ID
Product Name
Price
Stock

Fig 2.1: Product Entity Structure

The Tenant ID is used to identify a specific product to a specific vendor. All requests are first performed in the context of their tenant, and then only in that tenant's context are database operations performed.

2. Presentation Layer

A template-injection front-end engine is used by the system. Rather than create a whole new front-end for each vendor:

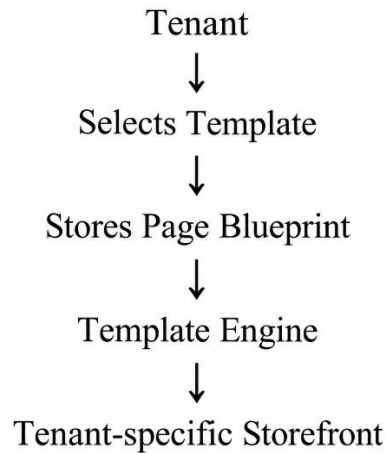


Fig 2.2: Tenant-Specific Storefront Generation Process

There are a limited number of designs (typically 4-5), which are pre-designed and a tenant can choose from. Then, per tenant content is attached into the selected page structure during runtime.

3. Domain & Tenant Resolution

The system supports:

Subdomain:

brand1.VendorHood.com

brand2.VendorHood.com

and

Custom domain:

www.brand1.com

www.brand2.com

The system checks the Host header that is sent in. If it is a platform subdomain, the tenantkey will be extracted from the subdomain. If not, the domain is looked in a domain-alias registry for the proper tenant.

4. Commission-Based Payment

At checkout, the payment is divided between the platform and vendor. Conceptually:

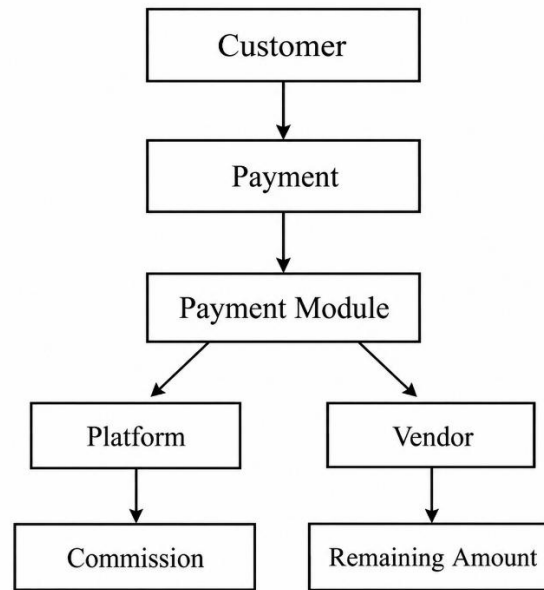


Fig 2.3: Commission-Based Payment Splitting and Settlement Flow

Our system proposes the money portion of the platform which is shared as:

Platform Share = $\text{Commission} \times \text{Transaction Amount}$

and the vendor's share as:

Vendor Share = $(1 - \text{Commission}) \times \text{Transaction Amount}$

Once the order is processed successfully, the payment gateway callback/webhook can be used to update the order status.

Overall Proposed System Flow

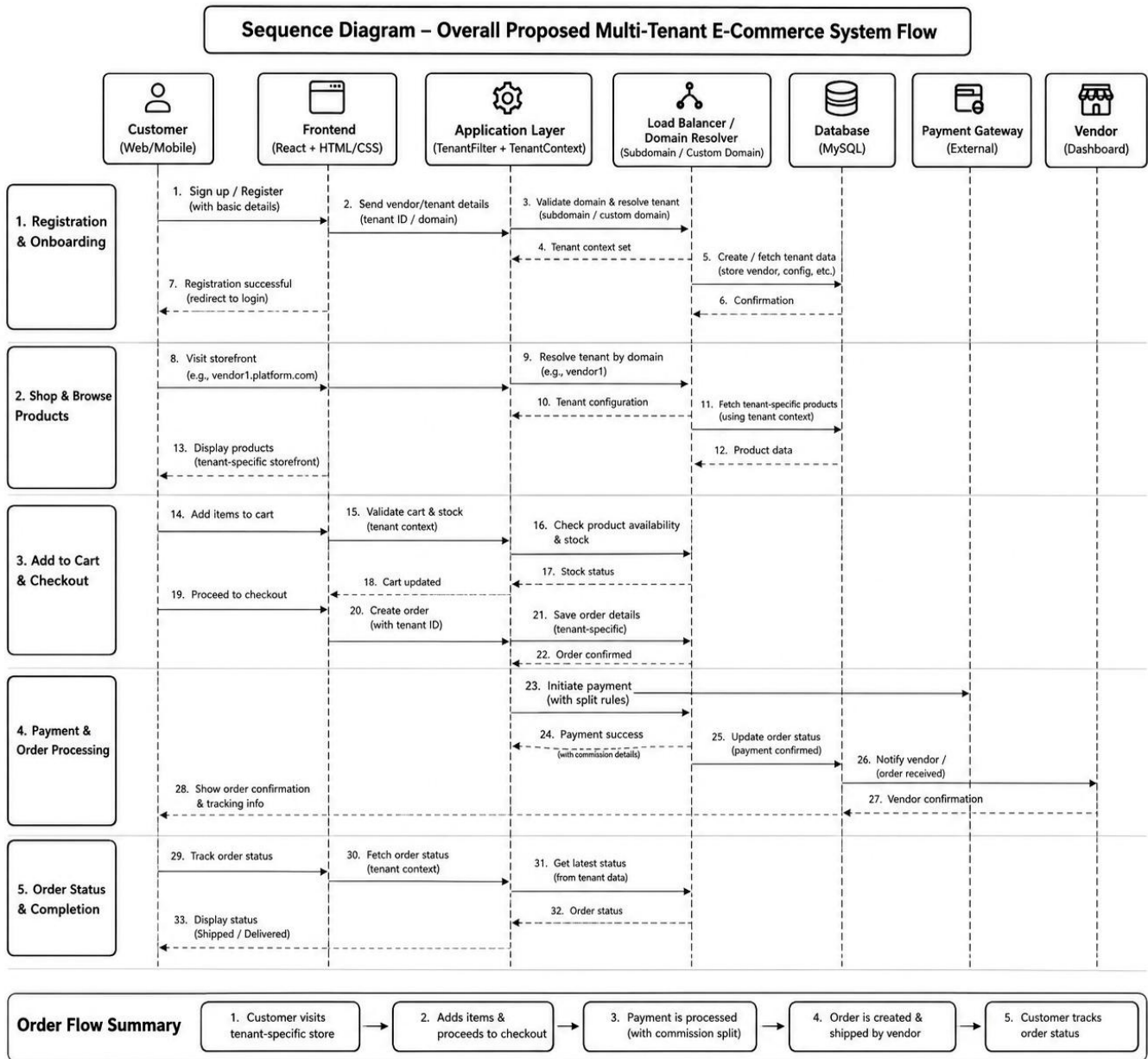


Fig 2.4: Overall Proposed Multi-Tenant E-Commerce System Flow

CHAPTER 3 - Requirement Gathering, Analysis and Planning

3.1 Requirement Specification

The idea for the Vendor Hood system is a -tenant Software-as-a-Service (SaaS) platform created for online retail vendors. The system offers a shared application setup that allows many vendors to run their online stores. Each vendor keeps their separate products orders, settings and payment details.

The needs, for the system came from the goals of the -tenant design the commission-based way of doing business that was explained in the research and the working version of the Vendor Hood system. The main needs include managing tenants, handling products creating stores identifying tenants by their domain, processing orders calculating commissions and keeping track of everything from an administrative view.

3.1.1 Functional Requirements

The functional requirements of VendorHood are presented below.

A. Vendor/Tenant Management

The system shall allow creation of vendor/tenant records, maintain a unique tenant identifier for each vendor, store vendor name and profile information, assign a subdomain to each vendor, support an optional custom domain for a vendor, associate a storefront template with each vendor, store a configurable commission rate for each vendor, allow administrators to view all registered vendors, allow administrators to update vendor information, and allow administrators to delete a vendor and its associated products and orders.

B. Product Management

The system shall allow vendors to add, update, and delete products, storing product name, price, description, image, and stock, along with additional attributes for different storefront categories. It displays only products belonging to the active tenant, while allowing administrators to view products across tenants where global administration is required.

The implementation uses a Product entity with a tenant identifier and product-specific attributes. Vendor-level product operations are exposed through the vendor controller, while global product access is provided through administrative operations.

C. Storefront Management

The system shall allow generating a storefront for the selected tenant, which will display its products tailored to fit tenant-specific information using the selected visual template of storefronts. It shall be possible to select from several predefined storefront templates, as well as change the storefront template within the same vendor, while still displaying its specific vendor branding (name and logo). It should be possible to reach the storefront via

subdomain-based URLs, as well as custom domain-based URLs.

The frontend currently provides five themes for storefronts: Studio Ivory Serif, Stark Dark Mono, Warm Editorial Sand, Stark Swiss Grid, and Heritage Botanical.

The selected theme will be stored as `themeld` in the data of the current vendor configuration, which may also be used in the frontend rendering logic.

D. Domain and Tenant Resolution

The system shall identify the active tenant from the incoming request.

The tenant-resolution process first checks for a tenant identifier, then checks the simulated host/domain information, followed by the actual Host header. It determines whether the host corresponds to a VendorHood subdomain and searches the vendor database using the subdomain; if no subdomain is found, it searches instead for a matching custom domain. The resulting tenant identifier is then assigned to the request context and used throughout subsequent database operations.

E. Order Management

The system shall enable the customers to place orders by the storefronts by association with the active tenant, storing the customer details, the ordered amount, the order status, and the date of the order. The system shall calculate the platform's commission and the respective vendor's share on each order, enable the said vendors to manage the status of their orders, and enable the customers to review their orders in the relevant storefronts. The Order entity shall contain fields for the tenant identifier, customer name, total amount of the order, platform commission, vendor's share, status, and the date.

F. Commission Calculation

The proposed business model contemplates the commission-based approach in which the value of the collected transaction is taken into consideration rather than a fixed subscription. For the order with the value of P and the coefficient of r , the platform's commission will be calculated as follows:

$$C = P \times \frac{r}{100}$$

The vendor share is:

$$V = P - C$$

where P = total transaction amount, r = vendor-specific commission percentage, C = platform commission, and V = vendor share.

The implementation performs this calculation in the storefront order-processing controller. For example, if:

$$P = 10,000$$

and

$$r = 8\%$$

then:

$$C = 10,000 \times 0.08 = 800$$

and:

$$V = 10,000 - 800 = 9,200$$

G. Administrative Management

The platform administrator shall be able to view, create, update, and delete registered vendors, view products and orders across tenants, and monitor total vendor count, total product count, total sales, and total platform commission.

The administrative functionality is implemented through AdminController.

3.1.2 Non-Functional Requirements

A. Multi-Tenant Isolation

Tenant isolation is a primary requirement of VendorHood.

Each tenant must access only its own products, orders, storefront information, and tenant-specific configuration.

B. Scalability

The architecture should support multiple vendors using a common application and database infrastructure. Shared infrastructure reduces duplication and allows additional tenants to be introduced without deploying a completely separate application instance for each vendor.

C. Performance

The system should provide acceptable response times when retrieving storefront information, products, orders, and vendor information.

The proposed system additionally identifies the noisy-neighbor problem as a concern in shared multi-tenant environments, particularly where tenants compete for common runtime resources.

D. Security

The system should prevent unauthorized cross-tenant access. Tenant identification must therefore occur before tenant-specific business and database operations.

E. Maintainability

The system should use modular components so that tenant management, product

management, order processing, storefront rendering, and administrative functionality can be modified independently.

F. Usability

The system should provide separate interfaces for the platform administrator, vendor, and customer.

The vendor interface provides inventory, order, profile, and storefront-management functionality.

3.2 Feasibility Study

The feasibility study determines whether the proposed VendorHood platform can be developed using the available technical resources and architecture.

3.2.1 Technical Feasibility

The project is technically feasible because the implementation uses established technologies for web application development.

The backend is implemented using Java 17, Spring Boot 3.3.2, Spring Web, Spring Data JPA, Hibernate, and MySQL Connector/J.

The frontend uses React 18, ReactDOM, JSX/Babel, Tailwind CSS, HTML5, CSS3, and JavaScript.

The database is MySQL 8.0 and is configured through Docker Compose.

The architecture also provides mechanisms for tenant context resolution and shared-schema tenant isolation, making the selected technology stack technically suitable for the proposed SaaS model.

3.2.2 Economic Feasibility

The proposed SaaS model is intended to reduce the fixed infrastructure burden on individual vendors by allowing multiple vendors to use shared application infrastructure.

Instead of requiring every vendor to independently maintain application infrastructure, database infrastructure, storefront software, development resources, and maintenance infrastructure, the SaaS platform provides a common platform that can serve multiple vendors.

The commission-only business model also links platform revenue to vendor transactions rather than requiring a fixed subscription payment.

The proposed system identifies this cost alignment as one of the primary motivations for the proposed architecture.

Note: The supplied project files do not provide an actual production cost analysis, hosting bill, or payment-provider pricing. Therefore, exact economic savings should not be claimed in this chapter.

3.2.3 Operational Feasibility

The system is both operationally feasible and the primary user activities match key online-retail workflows.

Platform Administrator : The platform administrator handles vendors, products, orders and platform statistics.

vendor : The vendor controls the store profile, products, stock, storefront template, orders and order status.

Customer: The customer uses the storefront to perform the actions of browsing the products, choosing products, ordering and obtaining information about orders.

The research architecture intentionally helps to separate logistics from the platform's scope of responsibility; order fulfilment and shipping are left to the vendor

3.2.4 Schedule Feasibility

The development process is broken into four phases of planned activities, with the goal of completing the project on time. The first phases consist of literature survey, requirement analysis, system architecture design and database design. These are followed by backend and frontend development. Once the core modules are completed, multi-tenancy and tenant-specific operations and commission-calculation features are added. The last step involves system integration, functional testing, performance evaluation, bug repair, and project documentation. The tasks are scheduled taking into account the duration of the project and are given to the team members in order to keep the project on time without delays. The exact dates and duration of the project will be adjusted based on the project's development records.

3.2.5 Security and Data Feasibility

Enforcing strict tenant-level data isolation is necessary for the shared-schema architecture so as to avoid unauthorized access between tenants. All database operations need to be performed within the appropriate tenant context. The requirement is met by a 'TenantFilter' within the system which determines the tenant from each incoming request and updates the tenant's 'TenantContext'. A 'TenantIdentifierResolver' is the service responsible for giving the tenant identifier, which is then used by Hibernate to get the tenant specific records. This way, tenants are identified centrally and data is isolated. The proposed system also emphasizes mis-scope of tenants as a significant security issue in multi-tenant systems that share a schema.

3.3 Methodology

The development methodology adopted for VendorHood is an iterative and modular development approach. This methodology is selected because VendorHood is a multi-tenant SaaS retail platform consisting of several interconnected functional components, including tenant management, vendor management, product and inventory management, customer management, order processing, payment processing, commission calculation, analytics, and administrative operations. Developing the entire system as a single unit would increase complexity and make it difficult to identify and resolve errors during development. Therefore, the system is decomposed into smaller and manageable modules that can be developed, tested, evaluated, and refined independently before being integrated into the complete platform.

3.3.1 Requirement Gathering

Requirements were identified from the problem of high fixed costs associated with conventional online-retail systems and from the architectural requirements of multi-tenant SaaS platforms.

The principal requirements identified were shared infrastructure, tenant isolation, vendor-specific storefronts, product management, order management, commission calculation, domain-based tenant identification, and administrative monitoring.

The research literature further identifies customization, tenant isolation, scalability, and resource utilization as important multi-tenant SaaS concerns.

3.3.2 System Design

The system design is based on four primary architectural components: shared-schema multi-tenancy, template-based storefront generation, domain-based tenant resolution, and commission calculation.

These components collectively form the core of VendorHood.

3.3.3 Implementation

The backend is implemented as a Spring Boot application. JPA/Hibernate is used for persistence, while MySQL provides the database layer.

The frontend provides the administration console, vendor portal, storefront interfaces, and template-based presentation layer.

3.3.4 Testing Strategy

Testing should be performed at multiple levels:

Unit Testing

Testing individual backend components and calculations.

Integration Testing

Testing communication between the frontend, backend, and database.

Tenant Isolation Testing

Testing whether a request for Tenant A can incorrectly retrieve data belonging to Tenant B.

Functional Testing

The Functional testing includes creation of Vendor, creation of Product, Updating Product, Deleting Product, creating Order, calculating Commission, Domain Resolution and Selection of Product Templates.

Performance Testing

Test system's behavior as the number of tenants, products, and orders grows.

3.4 Technology Stack

The following technologies are used in the VendorHood implementation.

Table 3.1: Technology Stack

Layer	Technology	Purpose
Frontend	React 18	Component-based user interface
UI Framework	Tailwind CSS	Responsive interface styling
JavaScript/JSX	JavaScript + JSX	Frontend application logic
JSX Compiler	Babel Standalone	Runtime JSX compilation in the required frontend
Backend	Java 17	Server-side application
Backend Framework	Spring Boot 3.3.2	REST API and application framework
Web Layer	Spring Web / Spring MVC	HTTP request and REST API handling
Persistence	Spring Data JPA	Database operations
ORM	Hibernate	Object-relational mapping and tenant handling
Database	MySQL 8.0	Persistent data storage
Database Driver	MySQL Connector/J	Java-MySQL connectivity
Containerization	Docker Compose	Local MySQL deployment
Build Tool	Maven	Backend dependency build management
Code Simplification	Lombok	Reduces Java boilerplate
Styling	CSS3	Custom cosmetics and UI styles

The pom.xml for the backend includes Java 17, Spring Boot 3.3.2, Spring Web, Spring Data JPA, MySQL Connector/J, Lombok and Spring Boot testing. React 18, ReactDOM, Babel and Tailwind CSS are loaded from CDNs on the frontend index.html.

3.4.1 Database Technology

MySQL 8.0 is used as the database management system. The Docker configuration sets up the VendorHood database and makes it available as a MySQL database on the host on port 3307 or on some other local port.

The key entities that are persistent are Vendor, Product and Order. Tenant identifiers are included in products and orders for the purposes of multi-tenant isolation.

3.5 Gantt Chart & Process Model

Project Activity		Week 6				Week 7				Week 8				Week 9				Week 10				Week 11				Week 12				Week 13				Week 14				Week 15			
		W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4				
1.	SAD & DD — Functional/Behavioral Models, Database, Architecture																																								
2.	Finalization of Platforms & Tools and Project Development																																								
3.	Presentation II & Evaluation — 60–70% Implementation																																								
4.	First Draft of Blue Book & Research Paper																																								
5.	Submission of Research Paper																																								
6.	Project Groups for Competitions																																								
7.	Demonstration of Complete Implementation																																								
8.	Presentation I & Evaluation																																								
9.	Presentation III & Evaluation — 100% Implementation																																								
10.	Research Paper Refinement																																								
11.	Blue Book Submission — Part I																																								

Fig 3.1: Gantt Chart

3.5.1 Development Process Model

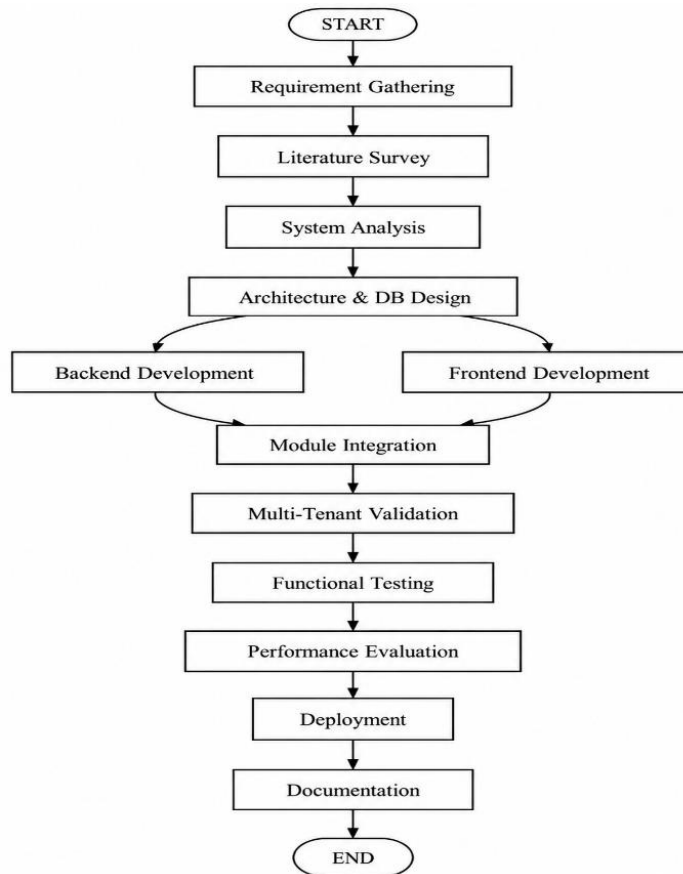


Fig 3.2: Development Process Model

3.5.2 Major Development Activities

Phase	Major Activities
Phase 1	Literature survey and problem identification
Phase 2	Requirement gathering and analysis
Phase 3	Multi-tenant architecture design
Phase 4	Database and entity design
Phase 5	Backend API development
Phase 6	Frontend and storefront development
Phase 7	Tenant isolation implementation
Phase 8	Commission calculation implementation
Phase 9	Domain resolution and template integration
Phase 10	Integration and testing
Phase 11	Performance evaluation
Phase 12	Documentation and final deployment

3.6 System Analysis

The VendorHood system is analyzed by three complementary models; namely functional model, structural model and behavioral model.

3.6.1 Functional Model

This is a model that shows the major operations that the system performs and how users interact with the system.

Major Actors

The main actors are the platform administrator, the vendor/tenant and the customer.

Platform Administrator Functions

The admin can handle, see, edit and remove vendors, see products and orders, and check sales and platform commission.

Vendor Functions

The vendor can view, add, update and delete products, edit their own profile, choose a storefront template, view orders and order status, update orders and order status, and view revenue information specific to the vendor.

Customer Functions

Customers use the storefront to access a tenant storefront, shop, place orders and view customer orders.

Use Case Representation

The functional relationship can be represented as follows:

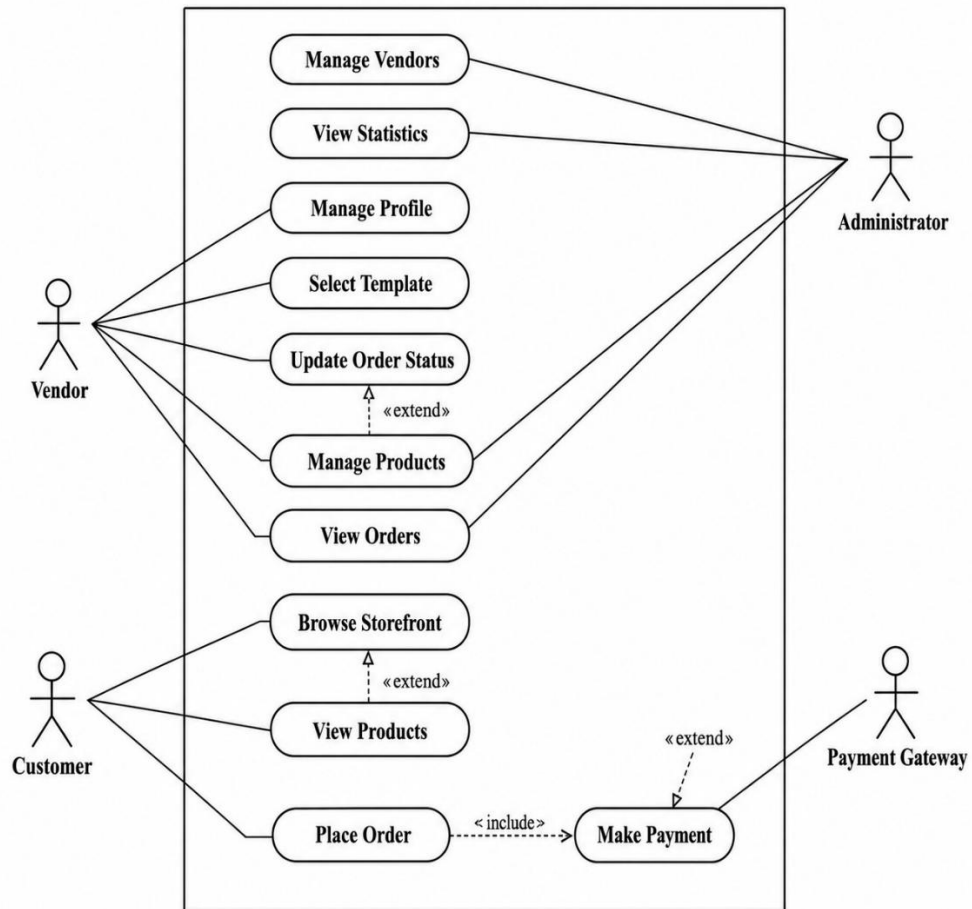


Fig 3.3: Functional Relationship

3.6.2 Tenant Isolation Structure

The tenant isolation architecture is one of the most important structural characteristics of VendorHood.

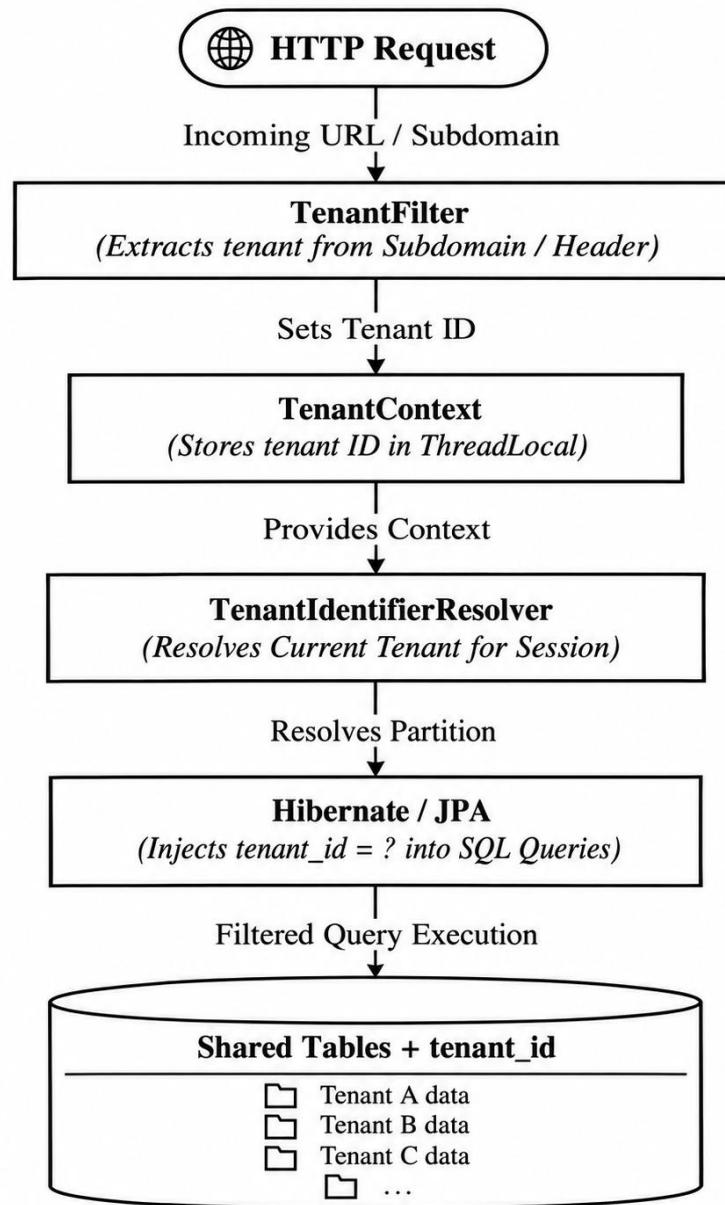


Fig 3.4: Tenant Isolation Structure

3.6.4 Behavioral Model

The behavioral model is a description of the behavior of VendorHood during normal operation.

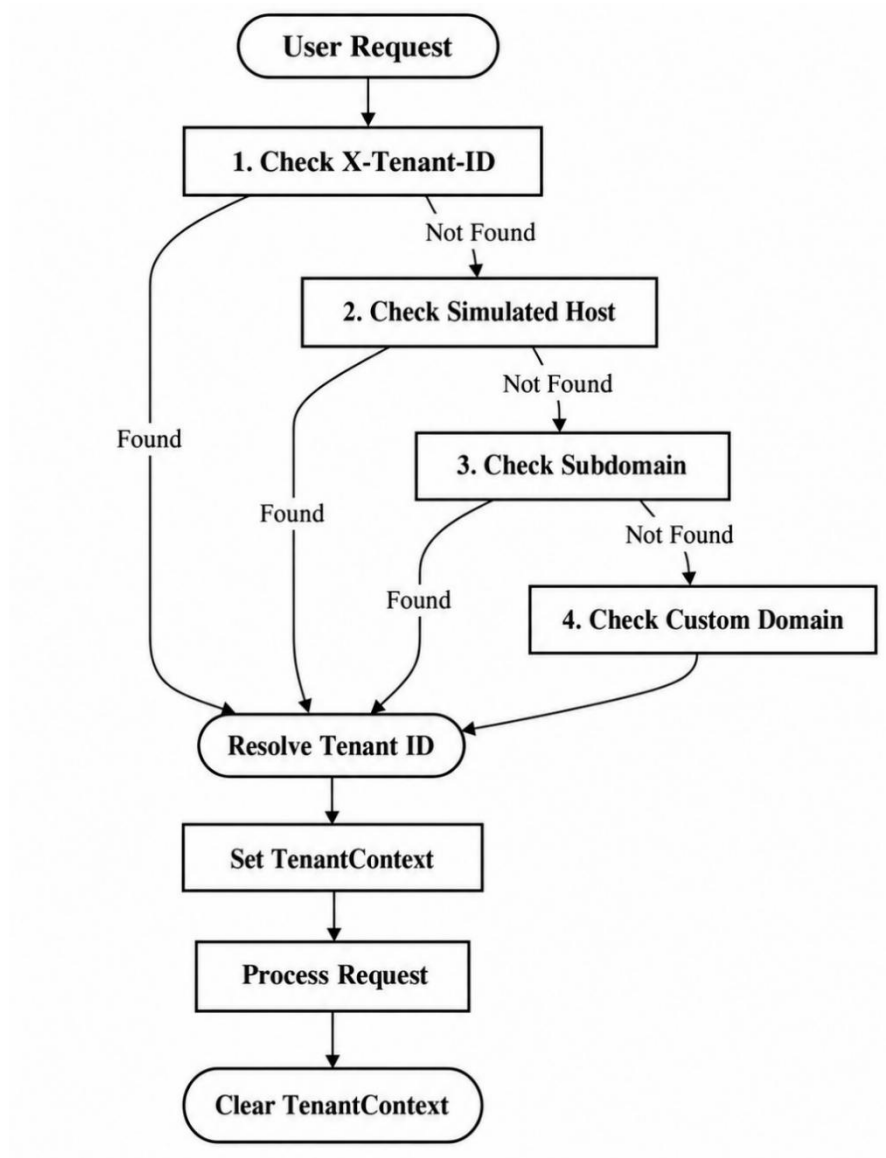


Fig 3.5: Behavioral Model

B. Order and Commission Processing

The order-processing behaviour is:

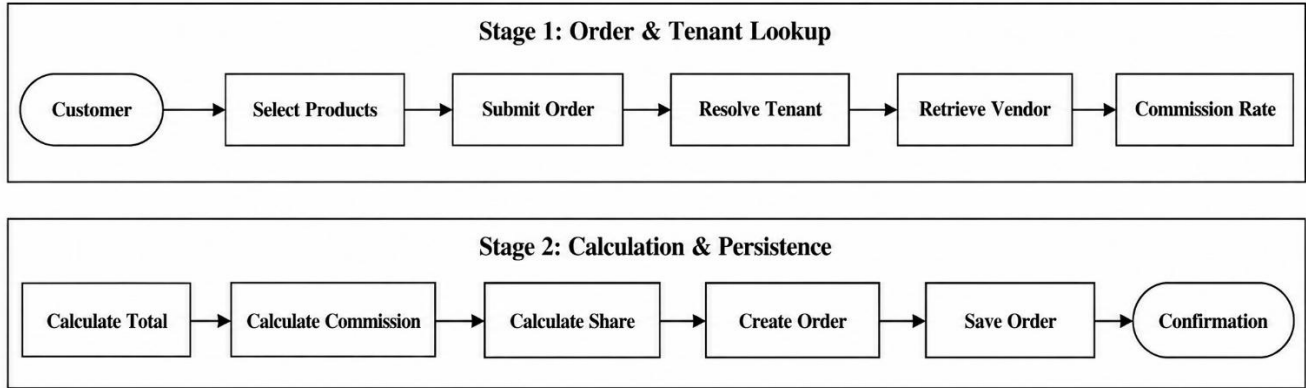


Fig 3.6: Order And Commission Processing

Mathematically:

$$C = P \frac{r}{100}$$
$$V = P - C$$

where C is the platform commission and V represents the vendor share. These calculations are currently done in `StorefrontController`, and saved in the order record.

3.6.5 Storefront Template Behavior

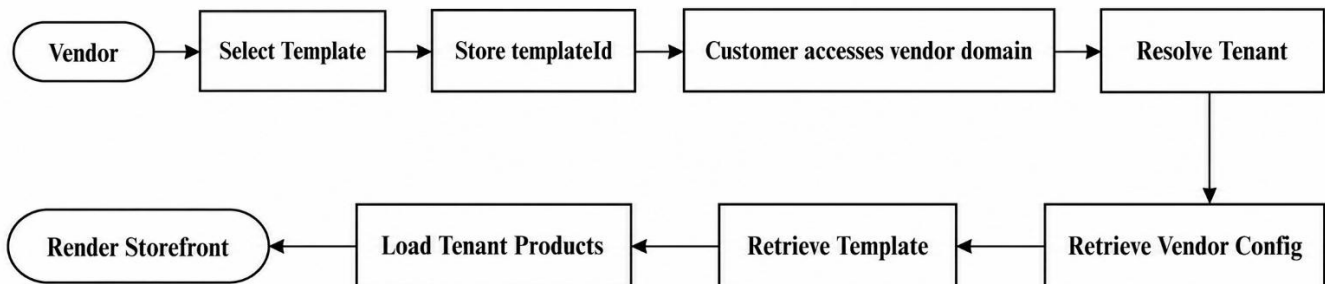


Fig 3.7: Storefront Template

Chapter 4 - System Design and Experimental Set up

The three-tier web architecture with a single underlining codebase and a centralized deployment model is the foundation of VendorHood. A single application instance supports multiple tenants (vendors) instead of multiple application instances per tenant (vendor). This not only simplifies deployment, reduces infrastructure needs and maintenance costs, but also enables multiple vendor storefronts to run independently from each other under the same platform. The presentation tier is realized in the form of a single page application (SPA) built with React, a JavaScript library for building web interfaces, which is delivered in a browser. The Customer Storefront allows customers to browse products and place orders, while the Vendor Portal provides vendors with features for managing their products, orders, and business operations. The Platform Admin Console gives you administrative capabilities to manage the Platform and the tenants in general. These consoles produce all requests to the application tier via standard HTTPS/JSON REST API calls.. The Public storefront requests may also describe what tenant the request is for in the request's `Host` header. A subdomain, for instance `kicks.vendorhood.com`, can be aliased to the vendor record. The implementation of the application tier is done with the Spring Boot framework and includes REST controllers like `AuthController`, `AdminController`, `VendorController`, and `StorefrontController`. The incoming requests are handled by a custom `TenantFilter` which decodes and validates the incoming authentication token, identifies the permissions of the user and then imposes access limitations according to the role. For instance, if a user is a `PLATFORM_ADMIN`, they're granted access to the `/api/admin/**` paths, and if they're a `VENDOR_ADMIN`, they're granted access to the `/api/vendor/**` paths. On successful authentication and authorization, the active tenant is determined by the token, the header `X-Tenant-ID` or for public storefront requests, by the header `Host`. Tenant identifier: It is stored in the internal `ThreadLocal` object of the `TenantContext` object for the entire duration of the request. This value is then passed to Hibernate, via `CurrentTenantIdentifierResolver`. Hibernate automatically filters ORM queries based on the derived tenant ID, when combined with the `@TenantId` annotation on entities like `Product` and `Order`. The data tier is used with shared database and shared schema (with a discriminator column like `tenant_id`) to separate tenant data. Thus, the physical deployment includes just one application server and database instance with the ability to support multiple vendor storefronts.

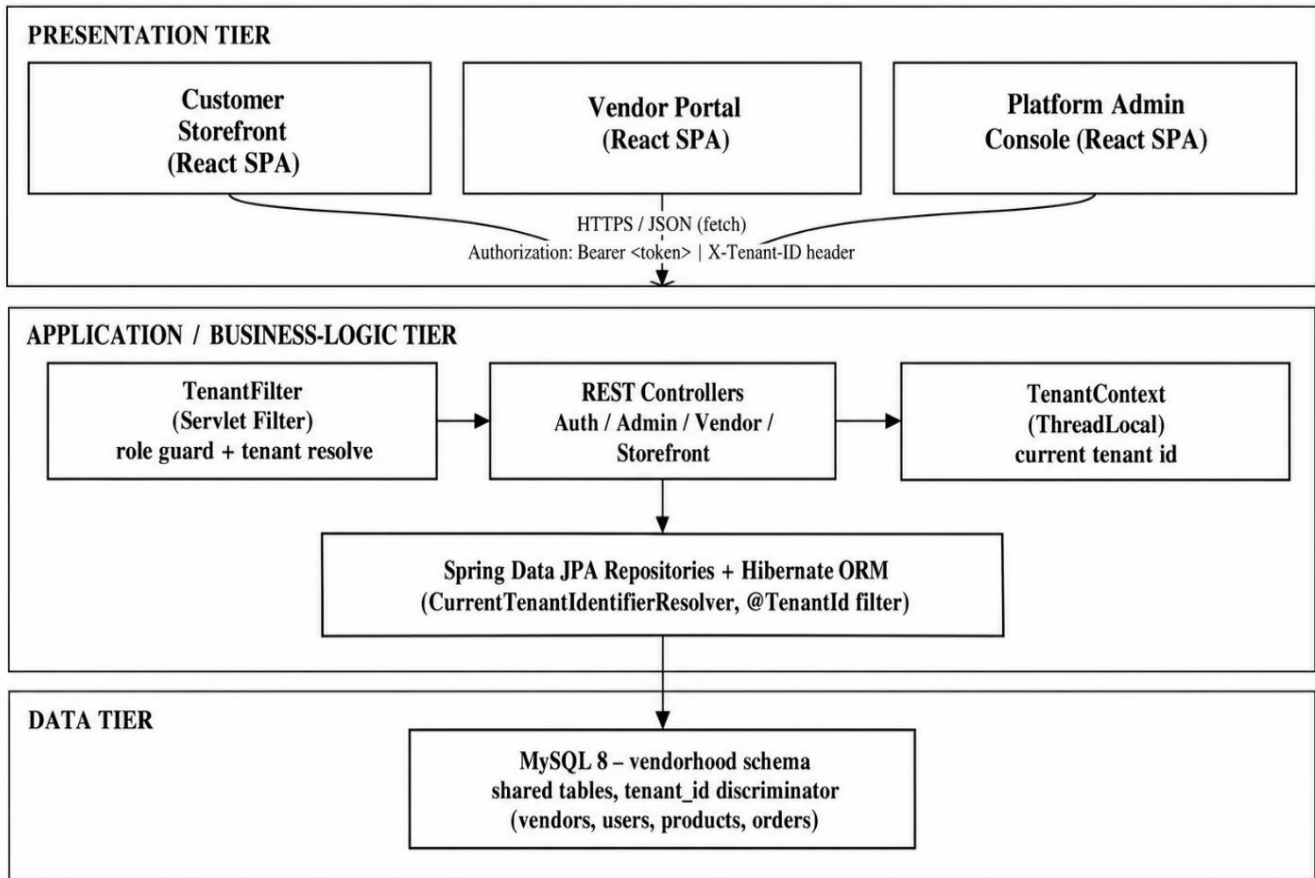


Fig. 4.1 — System architecture / physical layout block diagram of VendorHood.

4.2 Algorithm & Process Flow Design

Two process flows govern most of the platform's runtime behaviour: how a request is authenticated and routed to the correct tenant, and how a customer order is priced and split between the vendor and the platform. Both are implemented as straight-line procedural logic inside the Spring Boot backend rather than as external workflow engines, which keeps the request path short and easy to trace end to end.

4.2.1 Authentication and Tenant-Resolution Flow

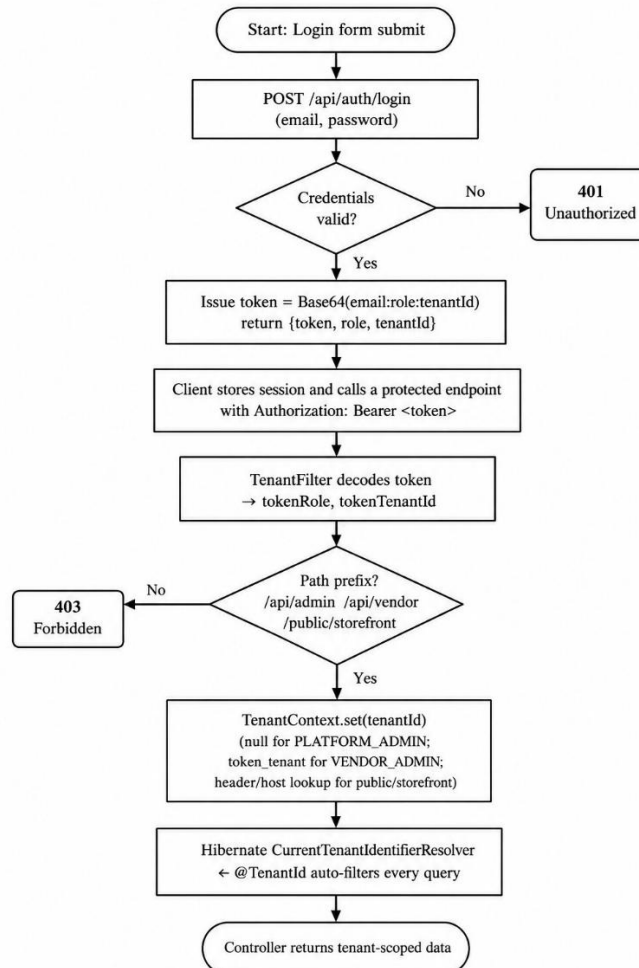


Fig. 4.2 — Flowchart: authentication and role-based tenant resolution.

Figure 4.2 traces a request from login through to a tenant-scoped response. `AuthController.login()` looks the submitted e-mail up in the users table and compares the stored password; on success it issues a lightweight bearer token built as `Base64(email:role:tenantId)` rather than a signed JWT. On every subsequent call, `TenantFilter` Base64-decodes this token to recover the caller's role and tenant id, applies the matching route guard, and — depending on whether the path belongs to the admin, vendor, or public/storefront namespace — either clears the tenant context (platform admin, sees every tenant), pins it to the token's own tenant (vendor admin, sees only their own store), or resolves it from the X-Tenant-ID header / Host header (public storefront traffic)— isolation is enforced once, centrally, in the filter and the ORM layer.

4.2.2 Checkout and Commission-Split Flow

Figure 4.3 shows the algorithm Storefront Controller. Create Storefront Order runs on every checkout. The customer's cart total and the target vendor's tenant id (taken from the resolved storefront) are posted to `/api/storefront/orders` with an `X-Tenant-ID` header identifying the vendor. The controller loads that `Vendor` row to read its per-tenant `commissionRate`, computes $\text{platformCommission} = \text{round}(\text{total} \times \text{rate} / 100, 2)$ and $\text{vendorShare} = \text{round}(\text{total} - \text{platformCommission}, 2)$, and persists a new `Order` row with status "Pending". The row is also stamped with the resolved tenant automatically, thanks to `Order's @TenantId`. Then the frontend fetches orders and products from the database again, to ensure that the customer's orders and the vendor's Sales tab match the new record.

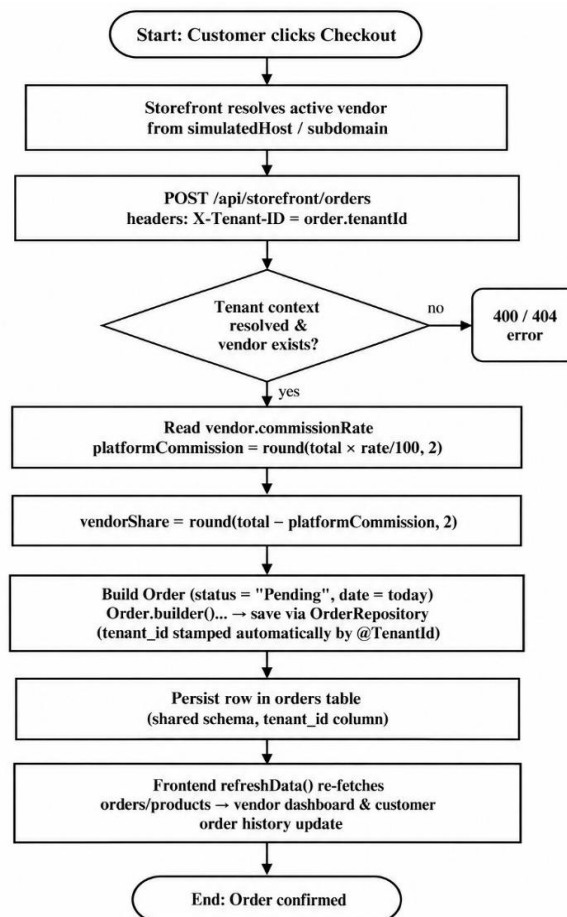


Fig. 4.3 — Flowchart: multi-tenant checkout and commission-split algorithm.

4.3 User Interface & Input Data Design

Figure 4.4 shows the structure of each of the four screens (as expressed in the component tree) with the regions marked with the data they read or write, where React is used as a view layer, with the Tailwind CSS utility classes styles.

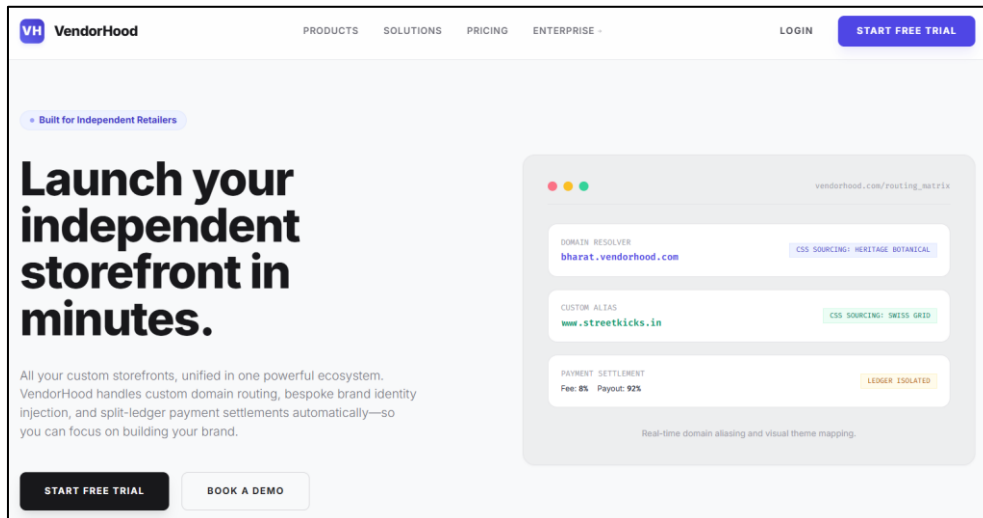


Fig. 4.4 — VendorHood Landing Page

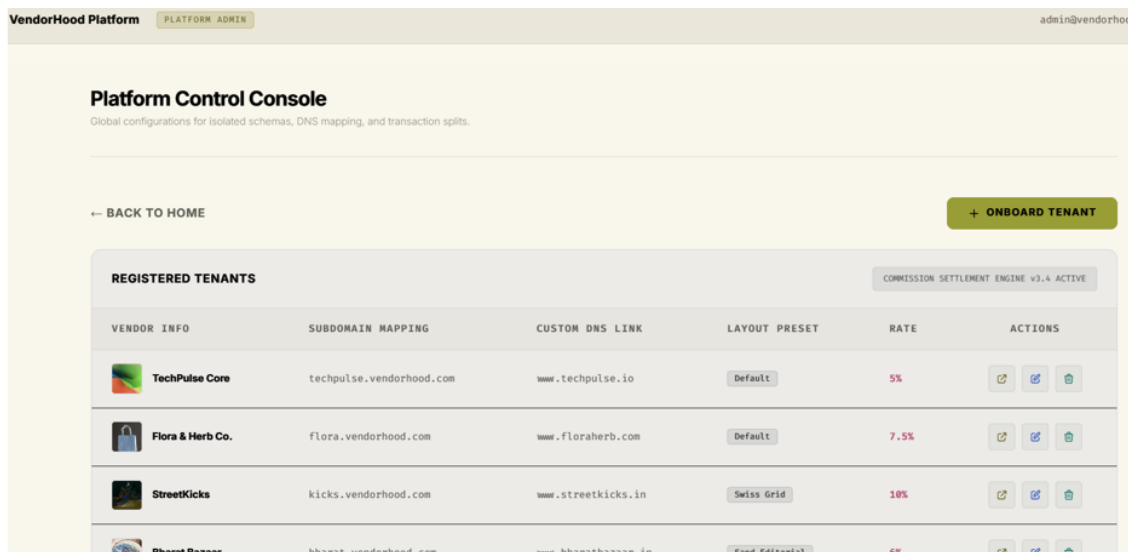


Fig. 4.5 — Tenant management List for VendorHood Admins

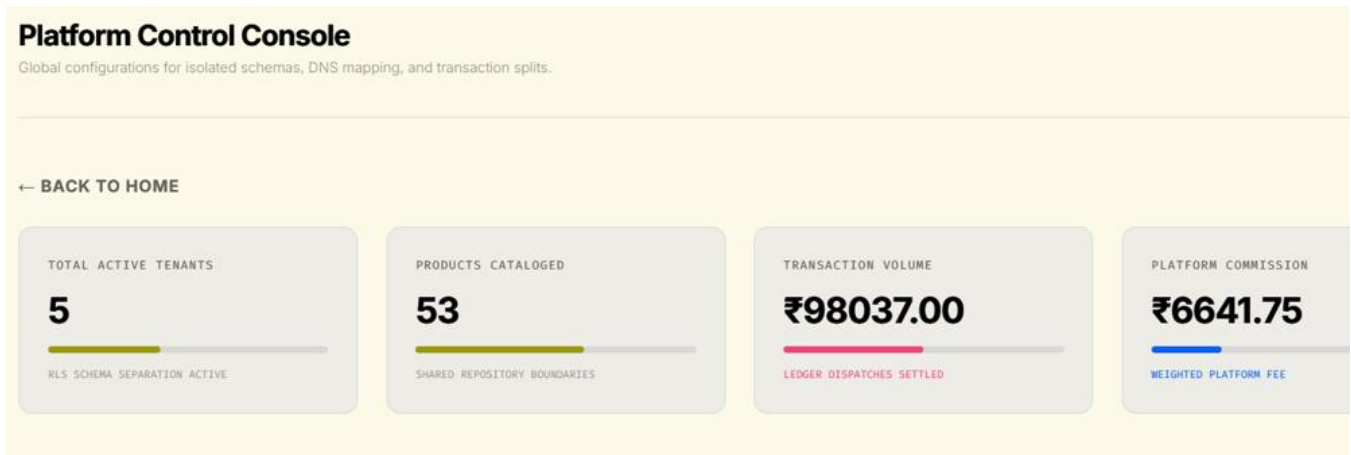


Fig 4.6 – Platform Analytics for VendorHood Admins

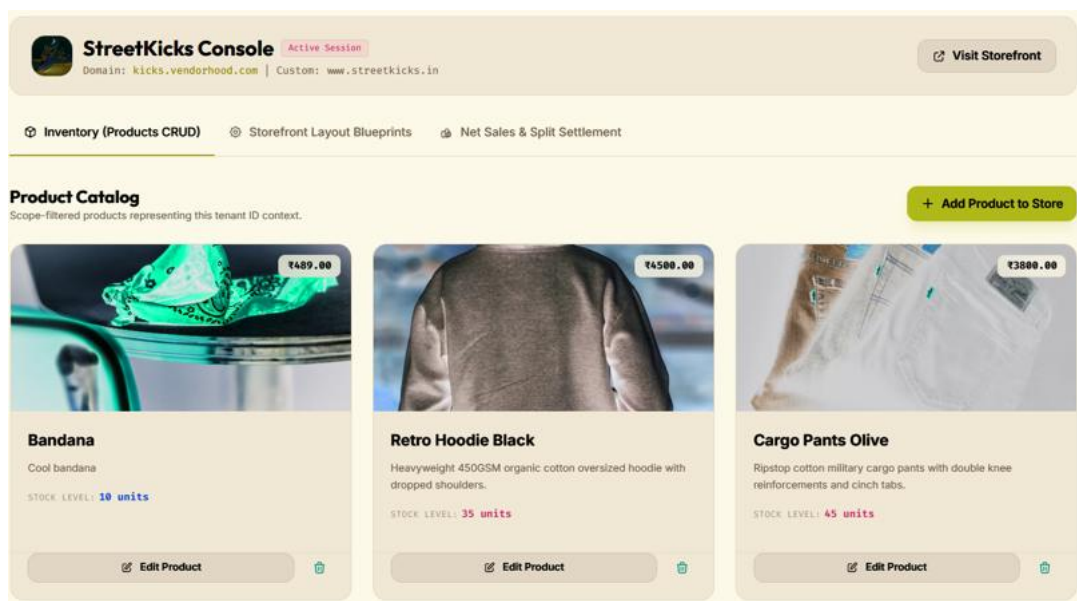


Fig. 4.7 — Product Catalog/ Inventory for Vendors

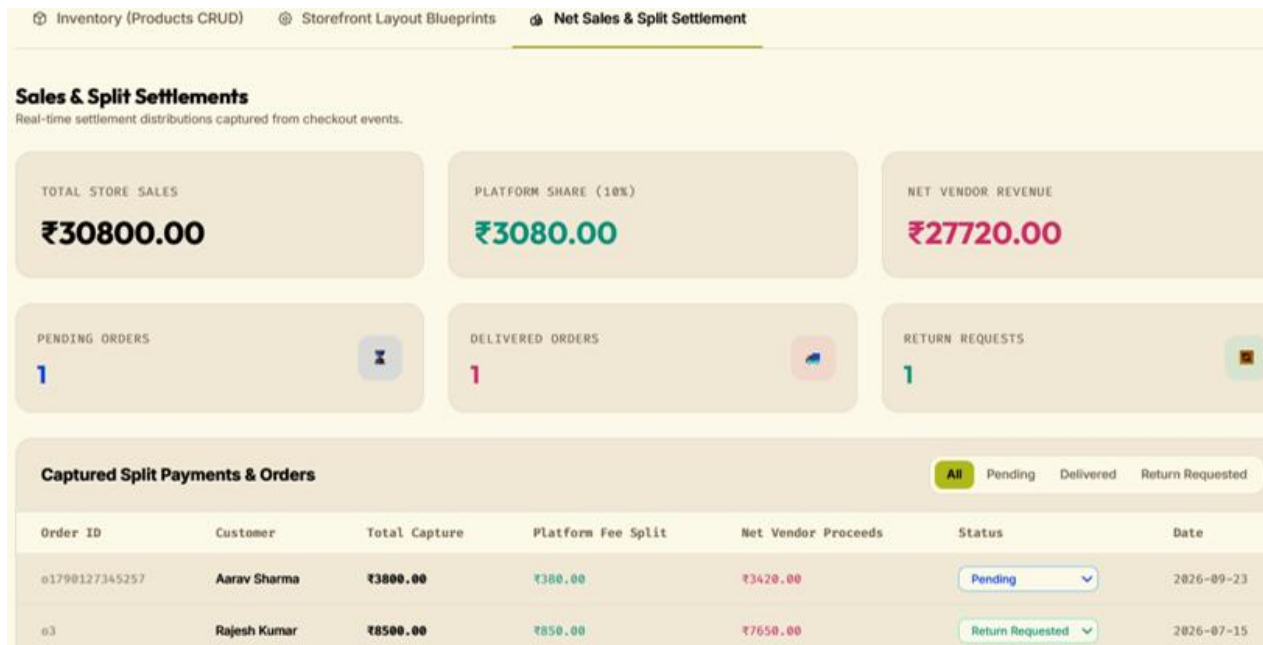


Fig. 4.8 — Shop Analytics+ Order Ledger for VendorHood Admins

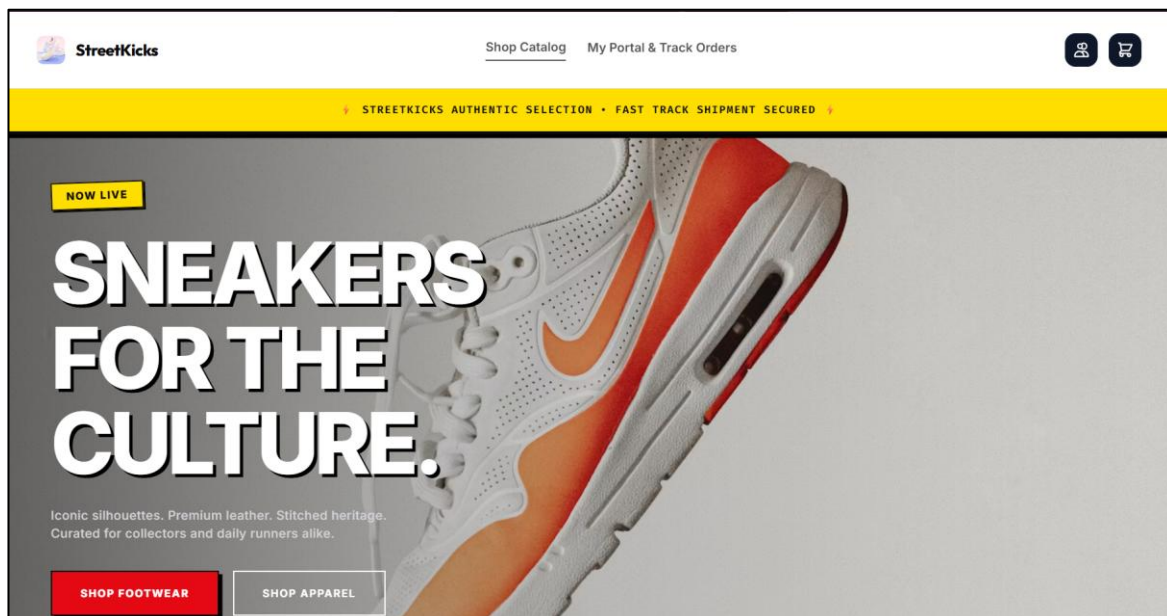


Fig. 4.9 — Vendor Customizable Storefront

The Login screen is the single entry point for all three roles: it posts an e-mail/password pair to /api/auth/login and branches into one of the other three screens based on the role returned. The Platform Admin console exposes global KPIs (vendor count, gross sales, platform commission) sourced from AdminController's aggregate queries, a vendor-management table for onboarding new stores, and read-only cross-tenant products/orders tables. The Vendor Portal is tabbed into Inventory (per-product create/edit/delete forms), Styling (storefront template selection) and Sales (revenue/commission KPIs plus an order-status control). The Customer Storefront renders the active vendor's catalogue inside whichever of the five visual templates is assigned to that tenant, and drives a cart drawer and a checkout modal that captures the customer's name and e-mail before the order is submitted. Table 4.1 summarises the input fields captured on each screen.

Table 4.1 — Input data design summary

Screen / Form	Input fields	Type / notes
Login	Email, Password	text / password, required; verified server-side against the users table
Add / Edit Vendor (Admin)	Name, Subdomain, Custom domain, Template ID (1–5), Commission rate (%), Logo URL	text / number; subdomain and custom domain are enforced unique at the database level
Add / Edit Product (Vendor)	Name, Price, Description, Image URL, Stock, plus template-specific fields (hover image, size preview, trending / limited / streetwear flags, origin, weight, cuisine)	text / number / boolean; price and stock are required numeric fields
Vendor Profile	Store name, Template ID, Logo URL, Custom domain	text / number, scoped to the logged-in vendor's own tenant row
Checkout (Customer)	Customer name, E-mail, Cart line items (product id, quantity), Coupon code (optional)	text / email; cart subtotal computed client-side, commission split computed server-side
Order status update (Vendor)	Status — Pending / Delivered / Return Requested	enum selected from a dropdown, sent as a PUT request

4.4 Experimental Setup and Tools

Table 4.2 - Software Requirements

Layer	Technology	Purpose
Backend	Java 17, Spring Boot 3.3.2 (Spring Web, Spring Data JPA)	REST API layer, dependency injection, embedded application server
Backend	Hibernate ORM (@TenantId, CurrentTenantIdentifierResolver)	shared-schema multi-tenant persistence
Backend	MySQL Connector/J	JDBC driver for MySQL
Backend	Project Lombok	boilerplate reduction (@Data, @Builder, constructors)
Backend	Apache Maven	build and dependency management (pom.xml)
Database	MySQL 8.0 (containerised)	relational store — one "vendorhood" schema shared by all tenants
Frontend	React 18 (development UMD build, via CDN)	component-based UI for all three consoles
Frontend	Babel Standalone	in-browser JSX-to-JS transpilation, no build step
Frontend	Tailwind CSS (Play CDN)	utility-first styling and per-tenant theme classes
Frontend	Google Fonts — Inter, Outfit, Playfair Display, Fira Code	typography for the five storefront templates
Tooling	Docker & Docker Compose	reproducible, disposable MySQL container
Tooling	Postman / browser DevTools	manual REST API testing
Tooling	Git	version control
IDE	IntelliJ IDEA / Visual Studio Code	backend and frontend development

(a) Development / hosting machine — used to build and run the full stack (Spring Boot backend + MySQL container):

Table 4.3 - Hardware Requirements

Component	Minimum	Recommended
Processor	Dual-core, 2 GHz	Quad-core, 2.5 GHz or better
RAM	4 GB	8 GB or more
Storage	~1 GB free (JDK, Maven cache, Docker image, DB data)	2 GB+ free on SSD
Operating system	Windows 10 / Ubuntu 20.04 / macOS 11 (64-bit)	any current 64-bit OS with Docker Desktop support
Required software	JDK 17, Docker Desktop, a modern web browser	same, plus an IDE (IntelliJ IDEA / VS Code)

(b) End-user machine — needed only to open and use the deployed website in a browser:

Table - 4.4 Implementation, Deployment and Testing

Component	Requirement
Device	Any desktop, laptop, tablet or smartphone capable of running a modern web browser
Processor / RAM	No requirement beyond what the browser itself needs (about 2 GB RAM is sufficient)
Browser	A current version of Chrome, Edge, Firefox or Safari with JavaScript enabled — the UI is a client-rendered React app compiled in-browser by Babel
Network	An active internet connection is required even after the page loads, to fetch React, Babel and Tailwind from their CDNs and the Google Fonts stylesheet, in addition to the app's own API calls
Display	Any screen size; the layout is responsive, though a width of 768 px or more gives the intended multi-column dashboard views
Installation	None — no plug-in or app download is needed to use the storefront, vendor, or admin consoles

4.5.1 Implementation

On the backend, the implementation follows a conventional Spring Boot layering of entity → repository → controller. The four JPA entities (User, Vendor, Product, Order) are written with Lombok's `@Data`, `@Builder`, `@NoArgsConstructor` and `@AllArgsConstructor` annotations to avoid hand-written getters, setters and constructors,

and Product/Order additionally carry Hibernate's `@TenantId` on their `tenantId` column so that the shared-schema isolation described in Section 4.1 is enforced without any explicit filtering code in the repositories. Spring Data JPA interfaces (`VendorRepository`, `ProductRepository`, `OrderRepository`, `UserRepository`) supply the standard CRUD methods plus a small number of custom finder methods (`findBySubdomain`, `findByCustomDomain`, `findByEmail`) and, in `ProductRepository` / `OrderRepository`, native `@Query` methods (`findAllProductsGlobal`, `sumTotalAmountGlobal`, `sumPlatformCommissionGlobal`, and similar) that deliberately bypass the Hibernate tenant filter so that `AdminController` can compute genuinely cross-tenant statistics for the platform-wide dashboard. `AdminController`'s vendor-deletion endpoint similarly falls back to raw `EntityManager.createNativeQuery()` calls to cascade-delete a vendor's products and orders, again because the entity-level `@TenantId` filter would otherwise hide rows belonging to any tenant other than the one currently in context.

The frontend was deliberately implemented without a Node build pipeline: `index.html` loads React 18, ReactDOM and Babel Standalone directly from CDN script tags, and each component file is included as a `<script type="text/babel">` tag that Babel transpiles in the browser at load time before `App.js` mounts the tree with `ReactDOM.createRoot()`. All cross-role UI logic — which console to show, which vendor is the "active tenant", and every CRUD call to the backend — is centralised in `App.js`, which holds `currentUser`, `currentView`, the `vendors/products/orders` arrays, and a `simulatedHost` value that stands in for a real wildcard-subdomain DNS setup during local development (`resolvedTenant` is derived from it with `useMemo` by matching against each vendor's subdomain or `customDomain`). Each of the five storefront visual themes is defined once as a small object of Tailwind class strings inside a `STOREFRONT_TEMPLATES` map, and two tenants in the seed data (`StreetKicks` and `Flora & Herb Co.`) additionally render through purpose-built `SneakerStorefront.js` / `SpiceStorefront.js` components, demonstrating the platform's template-injection approach to per-tenant branding without maintaining a separate codebase per vendor.

4.5.2 Deployment

The reference deployment is intentionally lightweight and single-host. `docker-compose.yml` brings up one `mysql:8.0` container (`vendorhood-mysql`) with a persistent named volume, exposing the database on host port 3307 to avoid clashing with any locally-installed MySQL instance. The Spring Boot backend is built and run with Maven (`mvn spring-boot:run`, or as the packaged JAR produced by the `spring-boot-maven-plugin`) and listens on port 8080; on startup, Hibernate's `ddl-auto=update` setting creates or updates the `vendors` / `users` / `products` / `orders` tables directly from the entity classes. The static frontend needs no separate web server for local use — `WebMvcConfig` maps every unmatched path to the parent project directory (`file:../`), so the same Spring Boot process that serves the API can also serve `index.html`, `style.css` and the component scripts; alternatively, the front-end files can be hosted from any static file server or CDN with the API base URL adjusted accordingly. For a production rollout beyond the current prototype, the natural next steps are to move the database credentials out of `application.properties` and into environment variables or a secrets store, replace the Base64 pseudo-token with a signed, expiring JWT, restrict `WebMvcConfig`'s CORS policy from `allowedOriginPatterns("*")` to the platform's real domains, replace the `CDN-plus-in-browser-Babel` frontend with a proper production build for performance, and put a reverse proxy or wildcard DNS record in front of the application so that each vendor's subdomain or custom domain resolves to the same backend the way `TenantFilter.resolveTenantByHost()` already expects.

4.5.3 Testing

For this stage of the project, testing was performed manually, as follows: A platform admin approving a new vendor, and verifying that the vendor appears correctly in the cross-tenant dashboard and the statistics; A vendor admin adding, editing and removing products, changing the storefront template, and moving an order from `Pending` to `Delivered` / `Return Requested`; and a customer navigating a vendor's storefront (changing the simulated host / subdomain), adding products to the cart, applying a coupon, and completing checkout. Multi-tenant isolation was verified explicitly by logging into two different vendor accounts in turn and confirming that `VendorController`'s product and order endpoints for one tenant never returned rows belonging to the other — the expected effect of the `@TenantId` / `CurrentTenantIdentifierResolver` pairing described in Section 4.1. Role-based access

control was checked by calling admin-only and vendor-only endpoints with the wrong role's token and confirming that TenantFilter returned HTTP 403, and public endpoints were checked without a token to confirm that tenant resolution still worked from the X-Tenant-ID header and the Host-header fallback. API responses — status codes and JSON payload shape — for every controller method were also spot-checked with Postman and the browser's network inspector, and the CDN-based UI was exercised in the latest Chrome and Firefox to confirm the Tailwind-driven layouts render consistently. The spring-boot-starter-test dependency is already present in pom.xml, so JUnit 5 / Spring Test-based automated unit and integration tests — particularly around the tenant-filter logic and the commission-split arithmetic — are a planned addition beyond this manual test cycle.

Chapter 5 — Results & Discussion

5.1 Outputs & Outcomes

The proposed system results in a multi-tenant SaaS system for online retail that enables multiple vendors to operate their individual storefronts using shared application infrastructure.

The major outputs and outcomes of the proposed system are:

1. Multi-Tenant Retail Platform

The System allows multiple vendors to operate independently on a common platform while maintaining logical separation of their data.

2. Tenant-Based Data Isolation

Each product and tenant-specific record is associated with a Tenant ID. Tenant context is established for incoming requests and is used to restrict database operations to the corresponding tenant. This helps prevent unintended cross-tenant data access.

3. Customizable Storefront

The system provides a template-injection front-end engine. Vendors can select predefined storefront templates, store the corresponding page blueprint, and generate a tenant-specific storefront without developing an entirely separate frontend.

4. Domain-Based Tenant Identification

The system supports tenant identification through both subdomains and custom domains. The incoming domain/Host header is used to determine the corresponding tenant.

5. Commission-Based Payment Processing

The proposed payment flow allows a transaction to be divided between the platform and vendor. The platform receives its commission while the remaining amount is allocated to the vendor.

6. Complete Retail Transaction Flow

The proposed workflow covers:

Vendor Registration → Tenant Creation → Tenant/Domain Assignment → Storefront Configuration → Customer Visit → Tenant Resolution → Login → Catalogue → Product Selection → Stock Verification → Checkout → Payment Split → Order Status Update → Order Confirmation.

Overall Outcome

The resulting system demonstrates how shared infrastructure, tenant isolation, configurable storefronts, domain resolution, and transaction-based monetization can be combined into a single SaaS-oriented retail architecture.

5.2 Analysis of Results & Interpretation of Data

The results can be analysed based on the major architectural objectives of the proposed system.

Tenant Isolation

The shared-schema approach allows multiple tenants to use the same database structure while distinguishing their records through a tenant identifier. This provides a resource-efficient architecture, but correct tenant-context enforcement is essential because improper filtering could result in cross-tenant data exposure.

Interpretation:

The architecture demonstrates that shared infrastructure can be used for multiple vendors, provided that tenant identification and data-access restrictions are consistently enforced.

Storefront Customization

The template-based approach separates storefront structure from tenant-specific content. Instead of creating a separate application for each vendor, predefined page blueprints can be populated with tenant-specific information.

Interpretation:

This reduces the amount of custom development required for individual vendors while still allowing different tenants to maintain distinct storefront presentations.

Domain Resolution

The system supports both platform subdomains and custom domains. The system determines the tenant based on the incoming domain and then establishes the corresponding tenant context.

Interpretation:

This allows individual vendors to have recognizable storefront addresses while continuing to operate on the shared SaaS infrastructure.

Payment Processing

The payment module separates the transaction into a platform commission and the remaining vendor amount.

Interpretation:

This supports the proposed transaction-based monetization model, where the platform's revenue is associated with successful retail transactions rather than requiring the vendor to bear the entire fixed cost of the platform.

Scalability and Performance

Because multiple tenants share application resources, the architecture can experience resource interference, particularly when one tenant generates significantly higher workloads than others. The system identifies this as the noisy-neighbor problem and discusses resource control and more isolated deployment approaches as possible solutions.

5.3 Discussion of Results & Limitations of the System

Discussion of Results

The proposed framework addresses the main problem identified in the research: the high fixed cost of establishing and operating independent online retail systems for small and micro-scale businesses. The combination of shared infrastructure, shared-schema multi-tenancy, tenant-level data isolation, template-based storefronts, subdomain/custom-domain resolution, and commission-based payment settlement provides an integrated architecture for SaaS-based online retail. The system also demonstrates an important trade-off: while sharing resources can reduce infrastructure costs, it increases the importance of tenant isolation, security, resource management, and performance control. Therefore, the system also considers alternative deployment approaches, including multi-tenant, multi-instance, and hybrid/microservice-oriented architectures.

Limitations of the System

1. Shared Resource Interference

Multiple tenants share common infrastructure. A tenant generating unusually high traffic or workload may consume more CPU, memory, or other resources and affect other tenants. This is referred to as the noisy-neighbour problem.

2. Complexity of Tenant Isolation

Because tenants use a shared database schema, incorrect tenant filtering or context handling could potentially expose data belonging to another tenant.

Therefore, tenant-context enforcement must be consistently applied throughout the application and data-access layers.

3. Limited Storefront Customization

The proposed template-injection approach relies on a predefined set of templates/page blueprints. Therefore, customization is more controlled than developing a completely independent storefront for every vendor.

4. Logistics Are Outside the Platform

The proposed system does not take responsibility for physical fulfilment and logistics. Vendors remain responsible for shipping and fulfilment activities.

5. Performance and Scalability Challenges

As the number of tenants and their workloads increase, resource contention can become more significant. The system therefore identifies resource isolation and microservice-level tenancy as areas requiring further consideration.

6. Payment Dependency

The transaction settlement process depends on the payment module/gateway and its callbacks or webhooks for updating transaction and order status.

Chapter 6 — Conclusion & Future Scope

6.1 Conclusion

This system presents a multi-tenant SaaS framework for online retail systems based on the principle that affordability and usability for small sellers are strongly influenced by how costs and risks are distributed over time. The system reviews the literature on conventional setup costs of software development and customization. Limited capital is a major challenge for businesses when it comes to infrastructure setup and continuous operational costs. To overcome this model, a multi-tenant architecture can be used, where multiple vendors share the same code and infrastructure, with logical isolation of data.

The proposed system is based on an online retail store and merges together some of the above-mentioned architectural components into a single system, using the multi-tenant approach. The approach is to implement shared-schema multi-tenancy with enforced tenant context to ensure logical separation of tenants and limit the risk of cross-tenant data access. Tenant-specific data is tied to a tenant identifier, and tenant context is set before business logic and data access operations are executed. This offers a systematic way of achieving data isolation in the shared environment.

Additionally, the system includes a front-end engine based on templates to enable vendors to develop individual storefronts without having to build them for each tenant. The tenant can choose from pre-designed layouts and component placement, and the desired layout and component configuration is maintained as tenant-specific data. At runtime, the rendering engine creates the appropriate storefront based on the chosen schema, content, and catalogue of the tenant. This allows for multiple rendering methods while keeping the same mechanism across all stores.

One of the other important components is domain-based tenant resolution, which is supported for subdomain-based storefronts as well as custom-domain-based storefronts. Requests can be checked using the Host header, either by a platform-defined subdomain or a custom domain alias that has been registered. Once the tenant is resolved, the tenant identifier becomes attached to the context of the request and is applied to limit future data operations to the correct tenant. This combines domain resolution, tenant identification, and data isolation functions into a single process.

The system further introduces transaction-dependent monetization through commission-based payment settlement. Instead of depending entirely on fixed charges, the platform can take a configurable percentage of a transaction at payment time. The payment process

divides the transaction between the vendor and the platform, with the platform receiving its commission and the vendor receiving the remaining amount. The settlement is also associated with order-status updates through payment callbacks, connecting payment processing with the subsequent order flow.

The System also recognizes that sharing infrastructure introduces important considerations related to performance isolation and resource management. In a shared Java environment, tenants may compete for CPU and memory resources, resulting in cross-tenant performance interference. The system therefore considers the importance of runtime observability and protection against the noisy-neighbor effect. It also discusses fine-grained resource control and the possibility of applying tenancy management at the service level when the system is divided into microservices.

The comparison of deployment approaches further considers multi-tenant, multi-instance, and hybrid microservice-based multi-tenant architectures in terms of operational cost, resource sharing, number of instances, privacy, isolation, and application design. The hybrid approach combines characteristics of shared and dedicated models, while also introducing greater architectural complexity. These alternatives demonstrate that deployment decisions involve trade-offs between resource sharing, isolation, privacy, cost, and architectural complexity.

Overall, the proposed system provides an integrated architecture for SaaS-based online retail by combining shared infrastructure, tenant isolation, customizable storefronts, domain-based tenant resolution, commission-based settlement, and considerations for scalability and performance. The system deliberately keeps logistics outside its scope, leaving order shipment and physical fulfilment to individual vendors. Instead, it concentrates on the programmable core of the retail platform, including identity, storefront creation, transaction processing, tenant management, and scaling while preserving tenant isolation.

6.2 Future Scope

The future scope of the proposed system is closely related to the increasing role of automation and analytics in multi-tenant platforms. The literature indicates that tenant-facing customization is likely to become more automated, while analytic intelligence can become an important capability as unified platforms collect and process data across different activities. The proposed system provides a foundation for such extensions while retaining its common rendering engine and multi-tenant architecture.

One important direction is the use of automated storefront synthetic AI agents as a

continuation of the existing template-selection process. Multi-tenant builder systems can already be conceptualized around schema generation and rendering. Based on this approach, a generative agent may generate a valid page schema using natural-language constraints and populate component structures and content without violating the design system of the platform. In the context of retail tenants, these agents could act as an abstraction over the existing template-injection mechanism, reducing the cognitive effort involved in storefront creation while maintaining the common rendering engine of the system.

Another area of future scope is analytics and inventory forecasting intelligence. Once the system gathers event streams such as page views, product interactions, shopping-cart activity, and purchases, these data can support analytics and recommendation logic. Earlier multi-tenant retail work has already considered extensions involving the analysis of viewed and purchased products, while broader multi-tenant platform descriptions refer to cross-module analytics and predictive forecasting enabled by integrated data. For the proposed system, such analytics would be oriented toward actionable vendor requirements, including predicting stockouts, identifying high-velocity SKUs, and generating demand signals that can be further used for inventory planning and decision-making

Appendix A

Abbreviation and Symbols

1. **SaaS:** Software as a Service
2. **API:** Application Programming Interface
3. **REST:** Representational State Transfer
4. **JPA:** Java Persistence API
5. **ORM:** Object-Relational Mapping
6. **SQL:** Structured Query Language
7. **HTTP:** Hypertext Transfer Protocol
8. **HTTPS:** Hypertext Transfer Protocol Secure
9. **JSON:** JavaScript Object Notation
10. **HTML:** HyperText Markup Language
11. **CSS:** Cascading Style Sheets
12. **JSX:** JavaScript XML
13. **UI:** User Interface
14. **URL:** Uniform Resource Locator
15. **ID:** Identifier
16. **CDN:** Content Delivery Network
17. **JDBC:** Java Database Connectivity
18. **JDK:** Java Development Kit
19. **JVM:** Java Virtual Machine
20. **IDE:** Integrated Development Environment
21. **CRUD:** Create, Read, Update and Delete
22. **DFD:** Data Flow Diagram
23. **UML:** Unified Modeling Language
24. **CPU:** Central Processing Unit
25. **RAM:** Random Access Memory
26. **DNS:** Domain Name System
27. **JWT:** JSON Web Token
28. **CORS:** Cross-Origin Resource Sharing
29. **KPI:** Key Performance Indicator
30. **PWA:** Progressive Web Application
31. **SKU:** Stock Keeping Unit
32. **AI:** Artificial Intelligence
33. **DB:** Database
34. **DBMS:** Database Management System
35. **MVC:** Model-View-Controller
36. **Maven:** Apache Maven Build and Dependency Management Tool
37. **MySQL:** MySQL Relational Database Management System
38. **B2B:** Business-to-Business
39. **PaaS:** Platform as a Service
40. **REST API:** Representational State Transfer Application Programming Interface

Appendix B

Definitions

1. Multi-Tenancy - A software architecture in which multiple independent tenants or vendors use a common application and infrastructure while their data and configurations remain logically separated.
2. Tenant - An individual vendor or business operating its own online storefront through the VendorHood platform.
3. Tenant ID - A unique identifier associated with a tenant and used to distinguish that tenant's records from those belonging to other tenants.
4. Tenant Isolation - The mechanism used to ensure that one tenant can access only its own products, orders, configurations and other tenant-specific information.
5. Shared-Schema Multi-Tenancy - A multi-tenant database approach in which multiple tenants use the same database schema and tables, with tenant-specific records distinguished using a tenant identifier.
6. Tenant Context - The identity of the tenant associated with the current request. It is established before tenant-specific business and database operations are performed.
7. Tenant Resolution - The process of identifying which tenant a request belongs to using information such as the tenant ID, subdomain, custom domain or Host header.
8. TenantFilter - A server-side filter responsible for processing incoming requests, determining the caller's role and resolving the appropriate tenant before the request reaches the application logic.
9. TenantIdentifierResolver - A component used with Hibernate to determine the tenant associated with the current database operation.
10. Shared Infrastructure - A system architecture in which multiple vendors use common application, database and computing resources instead of maintaining separate infrastructure.
11. SaaS - A software delivery model in which application functionality is provided as a shared service to multiple users or organizations through a common platform.
12. Multi-Tenant SaaS - A SaaS architecture in which a single application infrastructure serves multiple independent tenants while maintaining logical separation of tenant data.
13. Database Persistence - The process of permanently storing application data in a database so that it can be retrieved later.
14. Platform Administrator - The user responsible for managing vendors, products, orders and platform-level statistics across the VendorHood system.
15. Storefront - The customer-facing online retail interface through which a particular vendor displays its products and receives orders.
16. Template Injection - The process of generating a vendor-specific storefront by combining a predefined page structure with the tenant's configuration, branding and product information.
17. Domain Resolution - The process of determining the appropriate tenant from a domain or subdomain used to access the storefront.
18. Subdomain - A domain component used to provide a vendor-specific address under the platform's primary domain, such as brand1.platform.com.
19. Custom Domain - A domain independently registered or configured for a vendor's storefront.
20. Host Header - An HTTP request header containing the host or domain through which a request reaches

the application. VendorHood can use it to identify a tenant.

21. Domain Alias - An alternate domain associated with a particular tenant and used for identifying that tenant.
22. Noisy-Neighbor Problem - A condition in a shared environment where one tenant's unusually high workload consumes common resources and can affect the performance experienced by other tenants.
23. Platform Commission - The portion of an order's transaction amount retained by the VendorHood platform according to the vendor's configured commission rate.
24. Payment Splitting - The process of dividing a completed transaction into the platform's commission and the vendor's share.
25. Payment Gateway - A payment-processing component through which transaction information and payment results are handled. The system uses payment callbacks or webhooks to update transaction and order status.
26. Webhook - A callback mechanism through which an external payment system can communicate a transaction or payment-status update back to the application.
27. Role-Based Access Control - An access-control mechanism in which permissions are determined according to the role of the user, such as platform administrator or vendor administrator.
28. Bearer Token - A token transmitted with an HTTP request to represent an authenticated user's session or authorization information.
29. REST API - An application interface that allows the frontend and backend to communicate through HTTP-based requests and responses.
30. Controller - A backend component responsible for receiving application requests and invoking the appropriate business operations.
31. Cross-Tenant Data Access - Access to data belonging to a tenant other than the tenant associated with the current request. Preventing unintended cross-tenant access is a major security requirement of VendorHood.
32. Discriminator Column - A database column used to distinguish records belonging to different tenants. In VendorHood, the tenant identifier serves this purpose.
33. Three-Tier Architecture - An architecture that separates the system into presentation, application and data-related layers.
34. Presentation Layer - The user-facing part of the system responsible for displaying interfaces and collecting user interactions.
35. Application Layer - The backend layer responsible for business logic, authentication, tenant resolution, order processing and other application operations.
36. Database Layer - The persistence layer responsible for storing and retrieving system data.
37. Babel - A JavaScript compiler used in the supplied frontend to transform JSX into JavaScript that can execute in the browser.
38. Docker - A containerization technology used to package and run application-related services consistently.
39. Maven - A build and dependency-management tool used for the Spring Boot backend.

Appendix C

REFERENCES

- [1] F. Shaikh and D. Patil, "SaaS based Multi-tenant E-commerce to reduce cost and improve resource utilization," 2015 IEEE 9th International Conference on Intelligent Systems and Control (ISCO), Coimbatore, India, 2015, pp. 1-6, doi: 10.1109/ISCO.2015.7282302.
- [2] Pushpan, Shinu. (2024). Multi-Tenant Architecture: A Comprehensive Framework for Building Scalable SaaS Applications. International Journal of Scientific Research in Computer Science, Engineering and Information Technology. 10. 1117-1126. 10.32628/CSEIT241061151.
- [3] Rathod, Rohit & Londhe, Omkar & Waghmare, Harshad & Varande, Ajay & Khedikar, Sarita. (2025). A Multi-Vendor SaaS website builder with Role-Based CRM. K. Elissa, "Title of paper if known," unpublished.
- [4] Kumar, Ritesh. (2020). Multi-Tenant SaaS Architectures: Design Principles and Security Considerations. 5. 49-62. 10.35629/8193-05024962.
- [5] Sumit Kalra and T. V. Prabhakar. 2018. Towards Dynamic Tenant Management for Microservice based Multi-Tenant SaaS Applications. In Proceedings of the 11th Innovations in Software Engineering Conference (ISEC '18). Association for Computing Machinery, New York, NY, USA, Article 12, 1–5. doi: 10.1145/3172871.3172882M.
- [6] Z. Zhuang, C. Tran, H. Ramachandra and B. Sridharan, "Ensuring High-Performance of Mission-Critical Java Applications in Multi-tenant Cloud Platforms," 2014 IEEE 7th International Conference on Cloud Computing, Anchorage, AK, USA, 2014, pp. 618-626, doi: 10.1109/CLOUD.2014.88.
- [7] F. Shaikh and D. Patil, "Multi-tenant e-commerce based on SaaS model to minimize IT cost," 2014 International Conference on Advances in Engineering & Technology Research (ICAETR - 2014), Unnao, India, 2014, pp. 1-4, doi: 10.1109/ICAETR.2014.7012861.
- [8] C. Bemagoni and S. B. Kare, "Cloud Based E-Commerce Platform for Multi-Tenant and Multi-Vendor," International Journal of Engineering Research & Technology (IJERT), vol. 3, no. 9, pp. 96–99, Sept. 2014, Art. no. IJERTV3IS090134.
- [9] Pinto, Hugo & Luz, J & Ramos de Oliveira, Ricardo & Lopes de Souza, Paulo &

Souza, Simone. (2016). A Systematic Mapping Study on the Multi-tenant Architecture of SaaS Systems. 10.18293/SEKE2016-068.

[10] Makki, Majid & Landuyt, Dimitri & Lagaisse, Bert & Joosen, Wouter. (2021). Thread-level resource consumption control of tenant custom code in a shared JVM for multi-tenant SaaS. Future Generation Computer Systems. 115. 351-364. 10.1016/j.future.2020.09.025.

[11] Ochei, Laud & Petrovski, Andrei. (2018). A Decision-based Model for Optimizing the Deployment of Cloud-hosted Application Components to Guarantee Multitenancy Isolation. International Journal of Intelligent Computing Research. 9. 890-901. 10.20533/ijicr.2042.4655.2018.0108.

[12] Anusuru, Anil. (2021). Transforming E-Commerce Platforms through Multi-Tenant Architectures and Microservices. International Journal of Communication Networks and Information Security. 13.